



**UNIVERSIDAD TECNOLÓGICA DE
PANAMÁ FACULTAD DE
INGENIERÍA DE SISTEMAS
COMPUTACIONALES
DEPARTAMENTO DE
COMPUTACIÓN Y SIMULACIÓN
DE SISTEMAS
CARRERA LICENCIATURA EN INGENIERÍA DE SISTEMA Y
COMPUTACIÓN HERRAMIENTA DE COMPUTACIÓN GRÁFICA**

Trabajo Grupal # 5

**Título del Taller: Fundamentos de
animación 2D**

Integrantes:

Rodríguez, Lysmarie 8-1010-386

Moreno, Dereck 8-1004-1420

Campine, Kevin 8-1028-1050

Aparicio, Cristhofer 9-765-102

Chauhan, Gulam 8-1043-2255

Profesor:

Ing. Samuel Jiménez

SEMESTRE I, 2026

INTRODUCCIÓN

En este trabajo realizamos un programa en OpenGL utilizando la librería GLUT, con el objetivo de dibujar y animar un avión en 2D. Para hacerlo usamos diferentes figuras básicas como polígonos, líneas y círculos, las cuales nos permitieron formar cada parte del avión, como el cuerpo, las alas, la cabina, la rueda y la hélice.

La idea principal del taller fue poner en práctica los temas vistos en clase sobre primitivas gráficas, colores, transformaciones y animación. Además, agregamos movimiento al avión mediante las teclas de dirección, lo que hizo que el programa no fuera solamente una imagen estática, sino una figura interactiva.

Durante el desarrollo del trabajo fuimos construyendo el avión por partes, organizando cada componente en funciones diferentes para que el código fuera más fácil de entender y modificar. También se agregó una hélice animada, la cual gira constantemente mientras el programa se ejecuta.

Código

```
#include <windows.h>
#include <GL/glut.h>
#include <math.h>

// =====
// VARIABLES GLOBALES
// =====

// Posición del avión en pantalla.
// Estas variables cambian cuando se presionan las flechas del teclado.
float posX = 0.0f, posY = 0.0f;

// Ángulo usado para animar la hélice del avión.
float anguloHelice = 0.0f;

// Constante PI utilizada para calcular círculos.
const float PI = 3.14159265f;

// =====
// FUNCIONES DE DIBUJO BASE
// =====

// Función que dibuja un círculo relleno.
// Se usa para el centro de la hélice y la rueda.
void circulo(float x, float y, float r) {
    glBegin(GL_POLYGON);
    for (int i = 0; i < 100; i++) {
        float ang = 2.0f * PI * i / 100;
```

```

        glVertex2f(x + cos(ang) * r, y + sin(ang) * r);
    }
    glEnd();
}

```

// Función que dibuja el contorno de un círculo.

// Sirve para darle estilo cartoon al avión.

```

void contornoCirculo(float x, float y, float r) {
    glBegin(GL_LINE_LOOP);
    for (int i = 0; i < 100; i++) {
        float ang = 2.0f * PI * i / 100;
        glVertex2f(x + cos(ang) * r, y + sin(ang) * r);
    }
    glEnd();
}

```

// =====

// FUNCIÓN PARA DIBUJAR LA HÉLICE

// =====

```

void dibujarHelice() {

```

```

    glPushMatrix();

```

// Rota la hélice usando el ángulo actualizado por el timer.

```

    glRotatef(anguloHelice, 0.0f, 0.0f, 1.0f);

```

// Dibuja dos aspas de la hélice.

```

    for (int i = 0; i < 2; i++) {

```

```

        glPushMatrix();

```

```

// Cada aspa se rota 180 grados respecto a la otra.
glRotatef(i * 180.0f, 0.0f, 0.0f, 1.0f);

// Color gris metálico del aspa.
glColor3f(0.74f, 0.79f, 0.82f);
glBegin(GL_POLYGON);
    glVertex2f(-0.02f, 0.0f);
    glVertex2f(-0.04f, 0.28f);
    glVertex2f(0.04f, 0.28f);
    glVertex2f(0.02f, 0.0f);
glEnd();

// Contorno negro del aspa.
glColor3f(0.11f, 0.12f, 0.13f);
glBegin(GL_LINE_LOOP);
    glVertex2f(-0.02f, 0.0f);
    glVertex2f(-0.04f, 0.28f);
    glVertex2f(0.04f, 0.28f);
    glVertex2f(0.02f, 0.0f);
glEnd();

glPopMatrix();
}

```

```

// Dibuja el perno central de la hélice.
glColor3f(0.93f, 0.71f, 0.44f);
circulo(0.0f, 0.0f, 0.05f);

```

```

// Contorno del perno.
glColor3f(0.11f, 0.12f, 0.13f);
contornoCirculo(0.0f, 0.0f, 0.05f);

```

```

    glPopMatrix();
}

// =====
// FUNCIÓN PARA DIBUJAR EL AVIÓN
// =====

void dibujarAvion() {
    glPushMatrix();

    // Permite mover todo el avión según posX y posY.
    glTranslatef(posX, posY, 0.0f);

    // Define el grosor de las líneas de contorno.
    glLineWidth(3.5f);

    // -----
    // ALA TRASERA
    // -----
    glColor3f(0.89f, 0.27f, 0.16f);
    glBegin(GL_POLYGON);
        glVertex2f(0.35f, 0.12f);
        glVertex2f(0.60f, 0.22f);
        glVertex2f(0.65f, 0.15f);
        glVertex2f(0.45f, 0.08f);
    glEnd();

    // Contorno del ala trasera.
    glColor3f(0.11f, 0.12f, 0.13f);

```

```

glBegin(GL_LINE_LOOP);
    glVertex2f(0.35f, 0.12f);
    glVertex2f(0.60f, 0.22f);
    glVertex2f(0.65f, 0.15f);
    glVertex2f(0.45f, 0.08f);
glEnd();

// -----
// CUERPO PRINCIPAL DEL AVIÓN
// -----
glColor3f(0.89f, 0.27f, 0.16f);
glBegin(GL_POLYGON);
    glVertex2f(-0.45f, 0.02f);
    glVertex2f(-0.45f, 0.14f);
    glVertex2f(-0.30f, 0.18f);
    glVertex2f(0.00f, 0.18f);
    glVertex2f(0.35f, 0.22f);
    glVertex2f(0.48f, 0.25f);
    glVertex2f(0.50f, 0.15f);
    glVertex2f(0.38f, 0.03f);
    glVertex2f(0.00f, -0.06f);
    glVertex2f(-0.25f, -0.06f);
glEnd();

// Contorno del cuerpo.
glColor3f(0.11f, 0.12f, 0.13f);
glBegin(GL_LINE_LOOP);
    glVertex2f(-0.45f, 0.02f);
    glVertex2f(-0.45f, 0.14f);
    glVertex2f(-0.30f, 0.18f);
    glVertex2f(0.00f, 0.18f);

```

```
    glVertex2f(0.35f, 0.22f);
    glVertex2f(0.48f, 0.25f);
    glVertex2f(0.50f, 0.15f);
    glVertex2f(0.38f, 0.03f);
    glVertex2f(0.00f, -0.06f);
    glVertex2f(-0.25f, -0.06f);
glEnd();
```

```
// -----
// CABINA DEL AVIÓN
// -----
glColor3f(0.36f, 0.54f, 0.66f);
glBegin(GL_POLYGON);
    glVertex2f(-0.25f, 0.17f);
    glVertex2f(-0.15f, 0.27f);
    glVertex2f(0.02f, 0.25f);
    glVertex2f(0.05f, 0.18f);
glEnd();
```

```
// Contorno de la cabina.
glColor3f(0.11f, 0.12f, 0.13f);
glBegin(GL_LINE_LOOP);
    glVertex2f(-0.25f, 0.17f);
    glVertex2f(-0.15f, 0.27f);
    glVertex2f(0.02f, 0.25f);
    glVertex2f(0.05f, 0.18f);
glEnd();
```

```
// Línea divisoria del cristal.
glBegin(GL_LINES);
    glVertex2f(-0.08f, 0.26f);
```



```

        glVertex2f(-0.08f, 0.18f);
    glEnd();

    // -----
    // ALA PRINCIPAL
    // -----
    glColor3f(0.89f, 0.27f, 0.16f);
    glBegin(GL_POLYGON);
        glVertex2f(-0.15f, 0.02f);
        glVertex2f(0.18f, -0.01f);
        glVertex2f(0.45f, -0.38f);
        glVertex2f(0.38f, -0.42f);
        glVertex2f(0.05f, -0.32f);
    glEnd();

    // Contorno del ala principal.
    glColor3f(0.11f, 0.12f, 0.13f);
    glBegin(GL_LINE_LOOP);
        glVertex2f(-0.15f, 0.02f);
        glVertex2f(0.18f, -0.01f);
        glVertex2f(0.45f, -0.38f);
        glVertex2f(0.38f, -0.42f);
        glVertex2f(0.05f, -0.32f);
    glEnd();

    // -----
    // RUEDA DEL AVIÓN
    // -----

    // Soporte de la rueda.
    glColor3f(0.11f, 0.12f, 0.13f);

```

```

glBegin(GL_LINES);
    glVertex2f(-0.20f, -0.06f);
    glVertex2f(-0.20f, -0.22f);
glEnd();

// Neumático.
glColor3f(0.20f, 0.24f, 0.26f);
circulo(-0.20f, -0.22f, 0.06f);

// Contorno del neumático.
glColor3f(0.11f, 0.12f, 0.13f);
contornoCirculo(-0.20f, -0.22f, 0.06f);

// -----
// HÉLICE ANIMADA
// -----
glPushMatrix();

// Ubica la hélice en la parte delantera del avión.
glTranslatef(-0.48f, 0.07f, 0.0f);

// Dibuja la hélice con rotación.
dibujarHelice();

glPopMatrix();

glPopMatrix();
}

// =====

```

```
// FUNCIÓN DISPLAY
```

```
// =====
```

```
void display() {
```

```
    // Define el color de fondo.
```

```
    glClearColor(0.96f, 0.93f, 0.89f, 1.0f);
```

```
    // Limpia la pantalla.
```

```
    glClear(GL_COLOR_BUFFER_BIT);
```

```
    // Dibuja el avión.
```

```
    dibujarAvion();
```

```
    // Intercambia los buffers para mostrar la imagen final.
```

```
    glutSwapBuffers();
```

```
}
```

```
// =====
```

```
// FUNCIÓN TIMER PARA ANIMACIÓN
```

```
// =====
```

```
void timer(int value) {
```

```
    // Cambia el ángulo de la hélice para producir rotación.
```

```
    anguloHelice -= 18.0f;
```

```
    // Evita que el ángulo crezca indefinidamente.
```

```
    if (anguloHelice < -360.0f) {
```

```
        anguloHelice += 360.0f;
```

```
    }
```

```
// Solicita redibujar la pantalla.
glutPostRedisplay();

// Vuelve a llamar al timer aproximadamente cada 16 ms.
// Esto permite una animación cercana a 60 FPS.
glutTimerFunc(16, timer, 0);
}
```

```
// =====
// FUNCIÓN DE TECLADO ESPECIAL
// =====
```

```
void teclado(int tecla, int x, int y) {
    float paso = 0.05f;

    // Permite mover el avión con las flechas del teclado.
    switch (tecla) {
        case GLUT_KEY_UP:
            posY += paso;
            break;

        case GLUT_KEY_DOWN:
            posY -= paso;
            break;

        case GLUT_KEY_LEFT:
            posX -= paso;
            break;

        case GLUT_KEY_RIGHT:
```

```

        posX += paso;
        break;
    }
}

// =====
// CONFIGURACIÓN INICIAL DE OPENGL
// =====

void init() {
    // Define la matriz de proyección.
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Define el sistema de coordenadas 2D.
    gluOrtho2D(-1.0, 1.0, -1.0, 1.0);

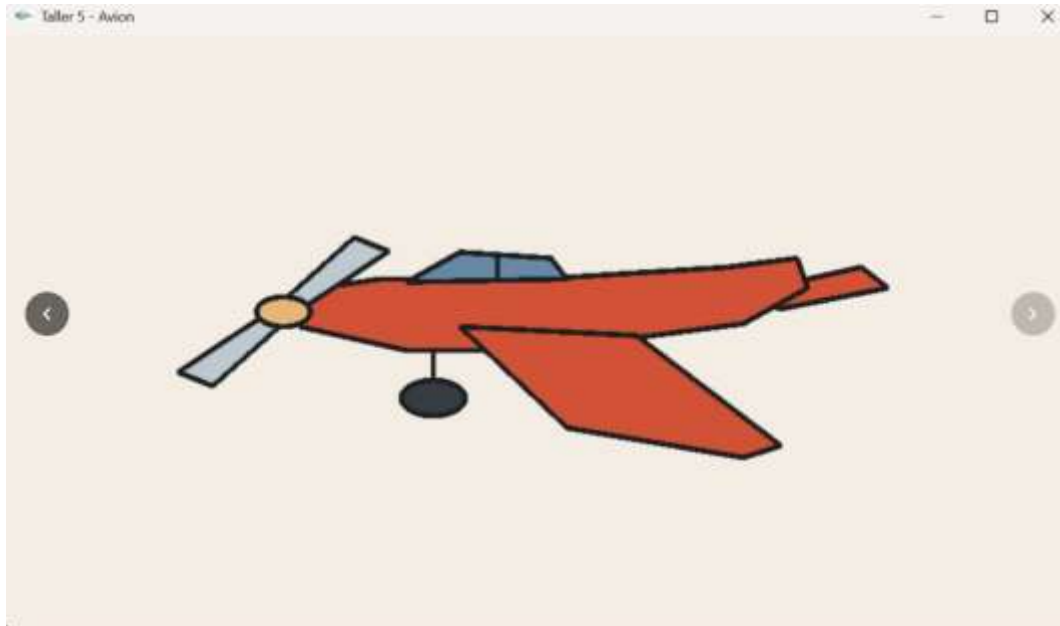
    // Cambia a la matriz de modelo/vista.
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    // Activa mezcla de colores y suavizado.
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    // Suaviza líneas y polígonos para mejorar el estilo visual.
    glEnable(GL_LINE_SMOOTH);
    glEnable(GL_POLYGON_SMOOTH);
    glHint(GL_LINE_SMOOTH_HINT, GL_NICEST);
}

```

```
// =====  
// PROGRAMA PRINCIPAL  
// =====  
  
int main(int argc, char** argv) {  
    // Inicializa GLUT.  
    glutInit(&argc, argv);  
  
    // Usa doble buffer y modo RGB.  
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);  
  
    // Define tamaño y posición de la ventana.  
    glutInitWindowSize(900, 500);  
    glutInitWindowPosition(150, 150);  
    // Crea la ventana del programa.  
    glutCreateWindow("Taller 5 - Avion");  
  
    // Llama a la configuración inicial.  
    init();  
  
    // Registra la función que dibuja la escena.  
    glutDisplayFunc(display);  
  
    // Registra la función para controlar las flechas del teclado.  
    glutSpecialFunc(teclado);  
  
    // Inicia el temporizador de animación.  
    glutTimerFunc(0, timer, 0);  
    // Inicia el ciclo principal de GLUT.  
    glutMainLoop();  
    return 0;  
}
```

Ejecución del programa



El avión tiene un estilo sencillo y caricaturesco, con el cuerpo de color rojo, la cabina azul, la hélice gris, una rueda en la parte inferior y contornos negros que ayudan a resaltar mejor la figura. La imagen demuestra que el programa se ejecutó correctamente y que se logró representar el avión usando las herramientas de OpenGL. También se puede apreciar que las partes del avión están bien ubicadas y que la escena se muestra dentro de una ventana gráfica.

Conclusión

Con este taller logramos aplicar varios de los conocimientos vistos en clase sobre OpenGL y GLUT. Aprendimos a construir una figura en 2D usando primitivas gráficas y a organizar mejor el código por medio de funciones. Esto nos ayudó a trabajar de una forma más ordenada, ya que cada parte del avión se podía modificar por separado.

Uno de los aspectos más importantes del trabajo fue la animación de la hélice, porque nos permitió entender mejor cómo usar un temporizador para actualizar constantemente la pantalla. También fue importante agregar el movimiento con el teclado, ya que eso hizo que el programa tuviera interacción con el usuario.